Pandas, Partie III

Plan

Pandas, Partie III

- **groupby**
- Tables croisées dynamiques (Pivot tables)
- Jointure de DataFrame
- Reshaping des données (melt, pivot)

Groupby.agg

Le **regroupement** permet de **résumer et d'analyser des tendances** en effectuant des opérations simultanées sur des sous-ensembles de données.

Étapes fondamentales :

- **Regrouper les lignes** selon une ou plusieurs colonnes clés.
  Exemple : regrouper toutes les lignes par année et mois.
- **Appliquer une fonction d'agrégation** sur chaque groupe.
  Exemple : calculer la somme du nombre de naissances pour chaque année et mois.

Applications

- Identifier des tendances dans les données
- Comparer des groupes entre eux
- Transformer des données détaillées en synthèses exploitables

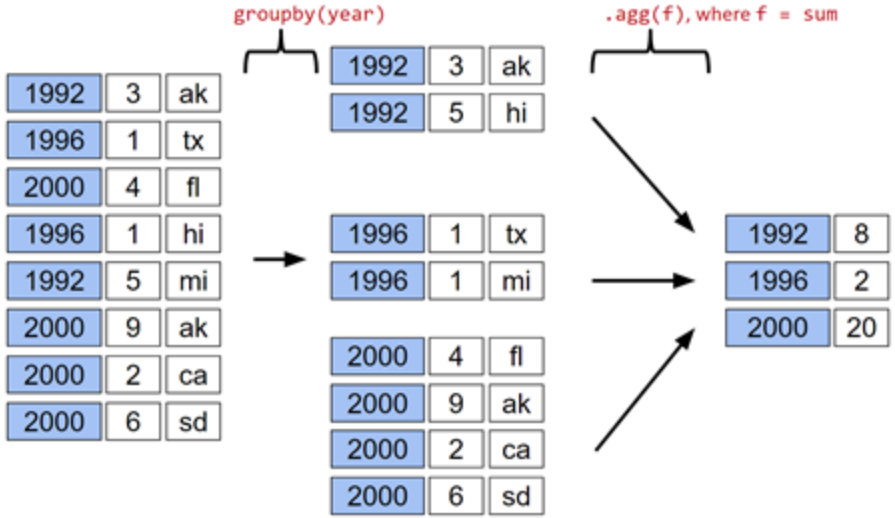
Groupby.agg

Une opération **groupby** suit le principe **split-apply-combine**:

- **Diviser (Split)** les données en groupes
- **Appliquer (Apply)** une fonction à chaque groupe
- **Combiner (Combine)** les résultats obtenus

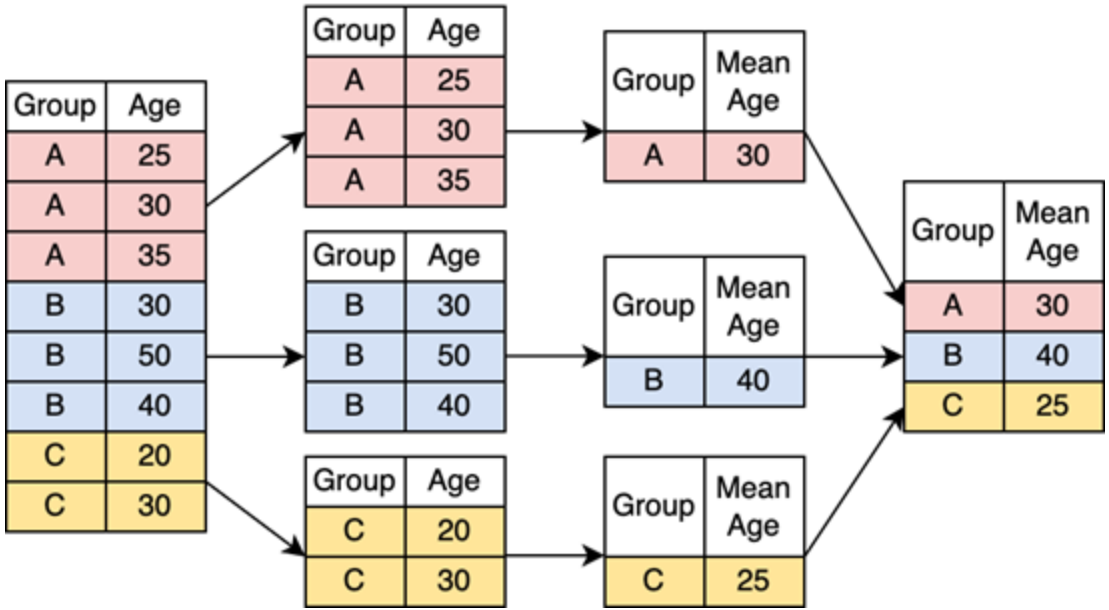
Exemple: `df.groupby("year").agg('sum')`:

Cette instruction regroupe les lignes du **DataFrame df** selon les valeurs de la colonne **"year"**, puis calcule la somme des colonnes numériques pour chaque année.

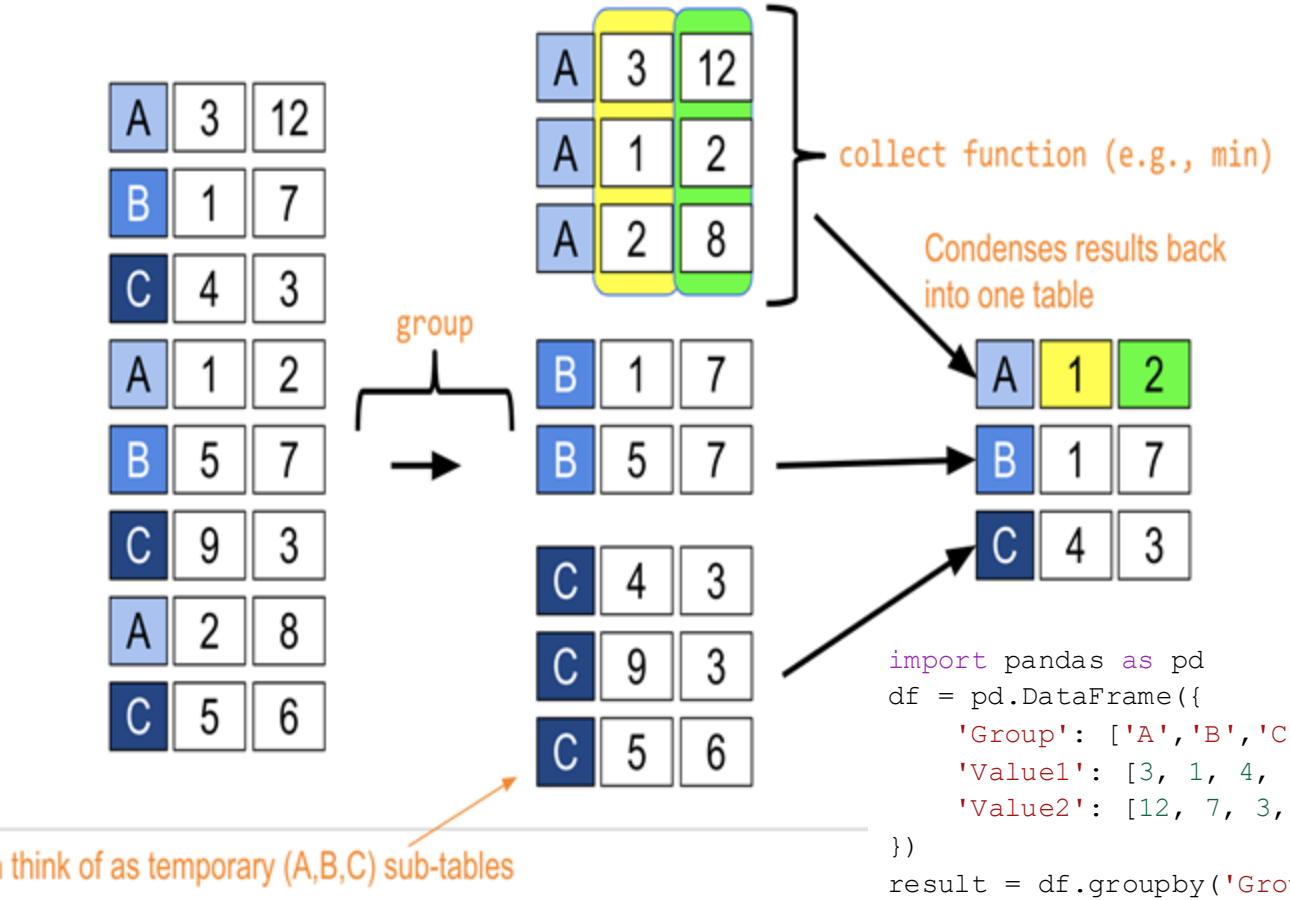


Split-Apply-Combine Paradigm

Step 1: Split Step 2: Apply Step 3: Combine



Exemple 1

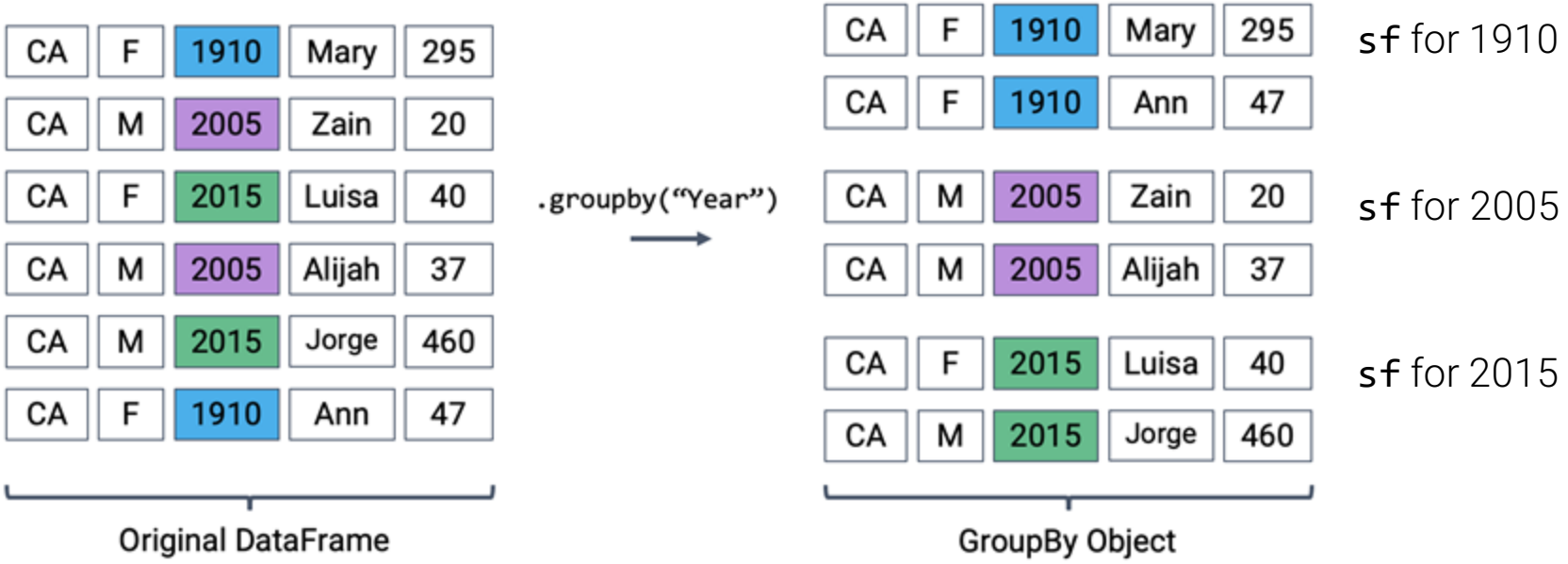



```
import pandas as pd
df = pd.DataFrame({
    'Group': ['A','B','C','A','B','C','A','C'],
    'Value1': [3, 1, 4, 1, 5, 9, 2, 5],
    'Value2': [12, 7, 3, 2, 7, 3, 8, 6]
})
result = df.groupby('Group').agg('min')
```

`.groupby()`

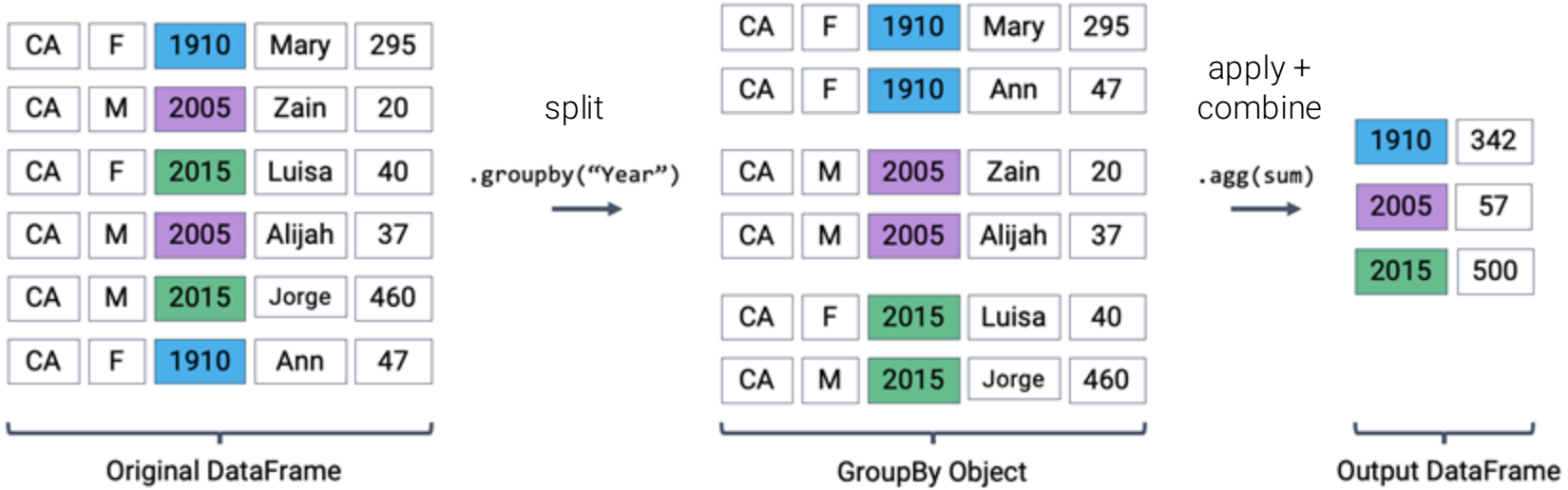
La méthode `.groupby()` partitionne un `DataFrame` (`df`) en plusieurs sous-DataFrames (« mini » DataFrames), notés **sf** (*subframes*).

- Chaque groupe correspond à **un sous-DataFrame** distinct.
- L'ensemble de ces **sous-DataFrames** est regroupé et représenté par un unique objet de type `DataFrameGroupBy`.



`.groupby()` and `.agg()`

La méthode `.agg()` **applique** une opération d'agrégation à chaque sous-DataFrame (`sf`), puis combine les résultats obtenus pour former un nouveau DataFrame.



Remarque: On travaille très rarement directement avec des objets `DataFrameGroupBy`. La méthode `.groupby()` doit pratiquement toujours être suivie d'une autre opération, telle que `.agg()` ou `.filter()`.

Question

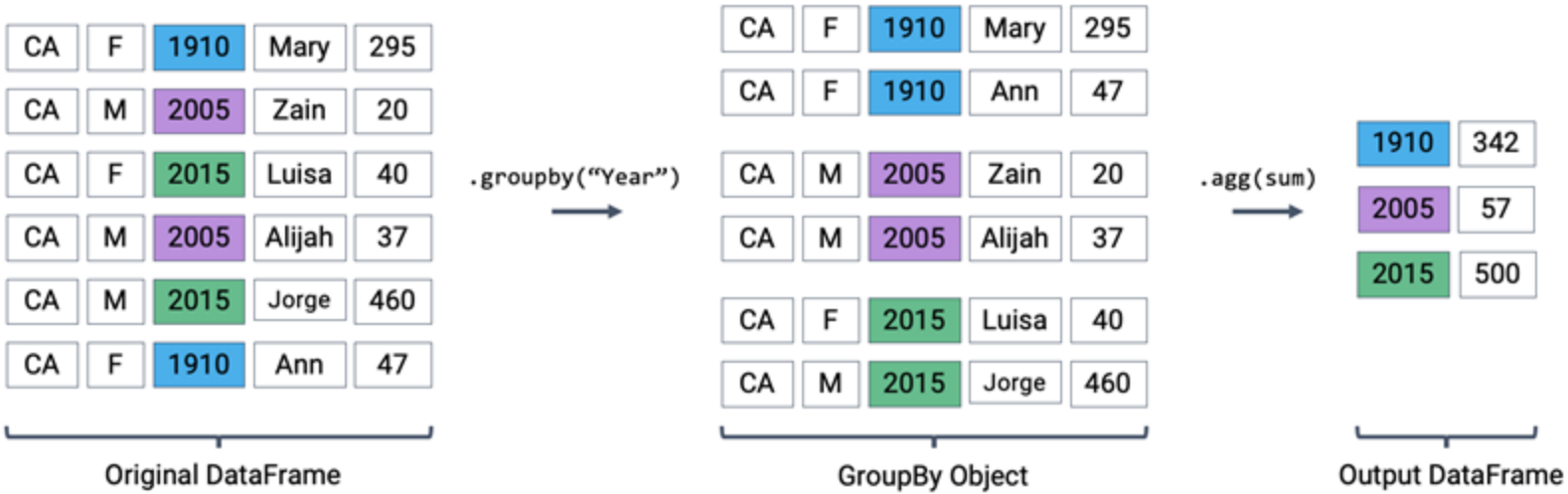
Syntaxe pour calculer le nombre total de bébés nés chaque année

```
babynames.groupby("Year")[["Count"]].agg('sum')
```

```
babynames.head()
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Ne sommer que la colonne **`Count`** et ignorer les autres colonnes. 🙈



Les fonctions d'agrégation

Que peut-on passer comme argument à la méthode `.agg( )`?

- Toute fonction qui agrège plusieurs valeurs en une seule valeur résumée.
- Exemples typiques:



Built-In Python Functions

```
.agg(sum)
.agg(max)
.agg(min)
```

NumPy Functions

```
.agg(np.sum)
.agg(np.max)
.agg(np.min)
.agg(np.mean)
```

Built-In **pandas** functions

```
.agg("sum")
.agg("max")
.agg("min")
.agg("mean")
.agg("first")
.agg("last")
```

Certaines fonctions d'agrégation peuvent être appelées directement, sans passer par `.agg( )` !

```
babynames.groupby("Year").mean()
```

↕️

```
babynames.groupby("Year").agg("mean")
```

Plotting Birth Counts

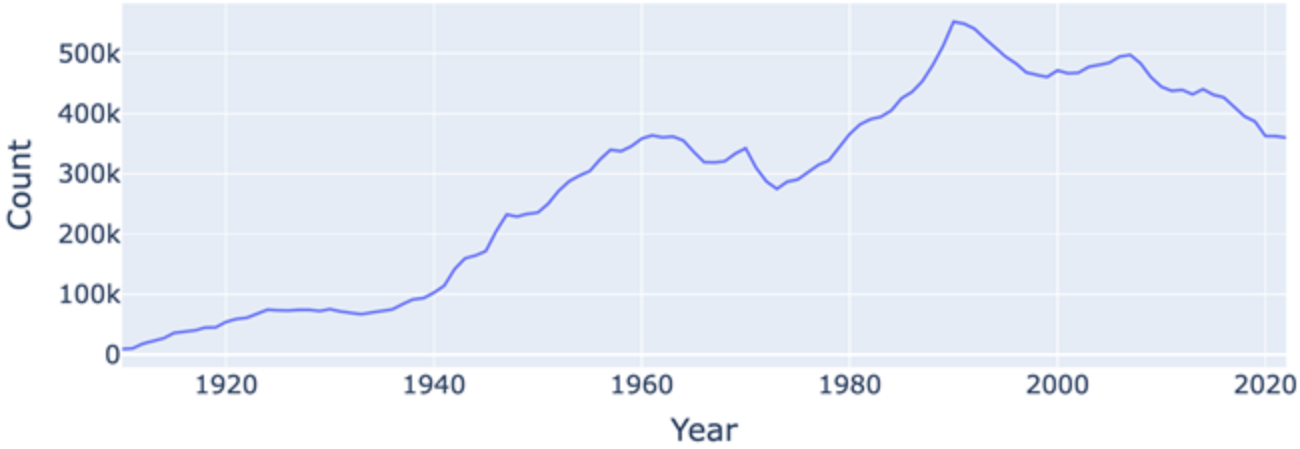
La représentation graphique des données *babynames*, agrégées par année, met en évidence une évolution particulièrement intéressante.

```
babies_by_year = babynames.groupby("Year")[["Count"]].agg(sum)
```

Year	Count
1910	9163
1911	9983
1912	17946
1913	22094
1914	26926
...	...
2018	395436
2019	386996
2020	362882
2021	362582
2022	360023

113 rows × 1 columns

```
import plotly.express as px
px.line(babies_by_year, y="Count")
```



Alternatives …

Différentes manières de créer des groupes pour chaque année :

```
babynames.groupby("Year")[["Count"]].agg(sum)
```

or

```
babynames.groupby("Year")[["Count"]].sum()
```

or

```
babynames.groupby("Year").sum(numeric_only=True)
```

Year	Count
1910	9163
1911	9983
1912	17946
1913	22094
1914	26926
...	...
2018	395436
2019	386996
2020	362882
2021	362582
2022	360023

113 rows × 1 columns

Exemple1

df =

col1	col2	col3
A	3	ak
B	1	tx
C	4	fl
A	1	hi
B	5	mi
C	9	ak
A	2	ca
C	5	sd
B	6	nc

df.groupby('col1').agg(**max**) =

A	??	??
B	??	??
C	??	??

Exemple1

Si aucune colonne n'est spécifiée, `.agg()` effectue l'agrégation sur toutes les colonnes non utilisées pour le groupement.

`df =`

col1	col2	col3
A	3	ak
B	1	tx
C	4	fl
A	1	hi
B	5	mi
C	9	ak
A	2	ca
C	5	sd
B	6	nc

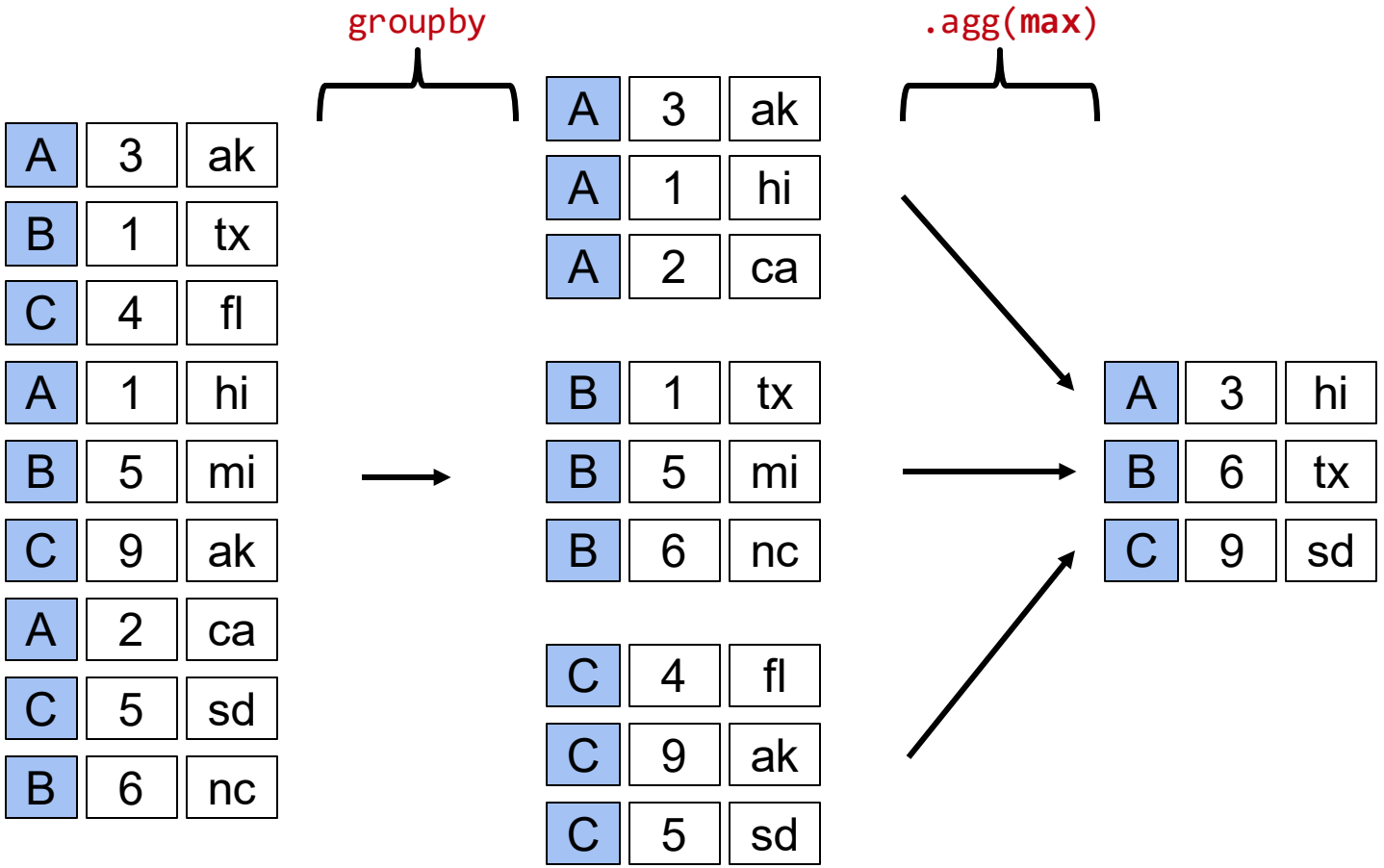
`df.groupby('col1').agg(max) =`



A	??	??
B	??	??
C	??	??

```
df.groupby('col1')[['col2','col3']].agg(max)
```

Exemple1


groupby
.agg(max)
A 3 ak
B 1 tx
C 4 fl
A 1 hi
B 5 mi
C 9 ak
A 2 ca
C 5 sd
B 6 nc
A 3 ak
A 1 hi
A 2 ca
B 1 tx
B 5 mi
B 6 nc
C 4 fl
C 9 ak
C 5 sd
A 3 hi
B 6 tx
C 9 sd

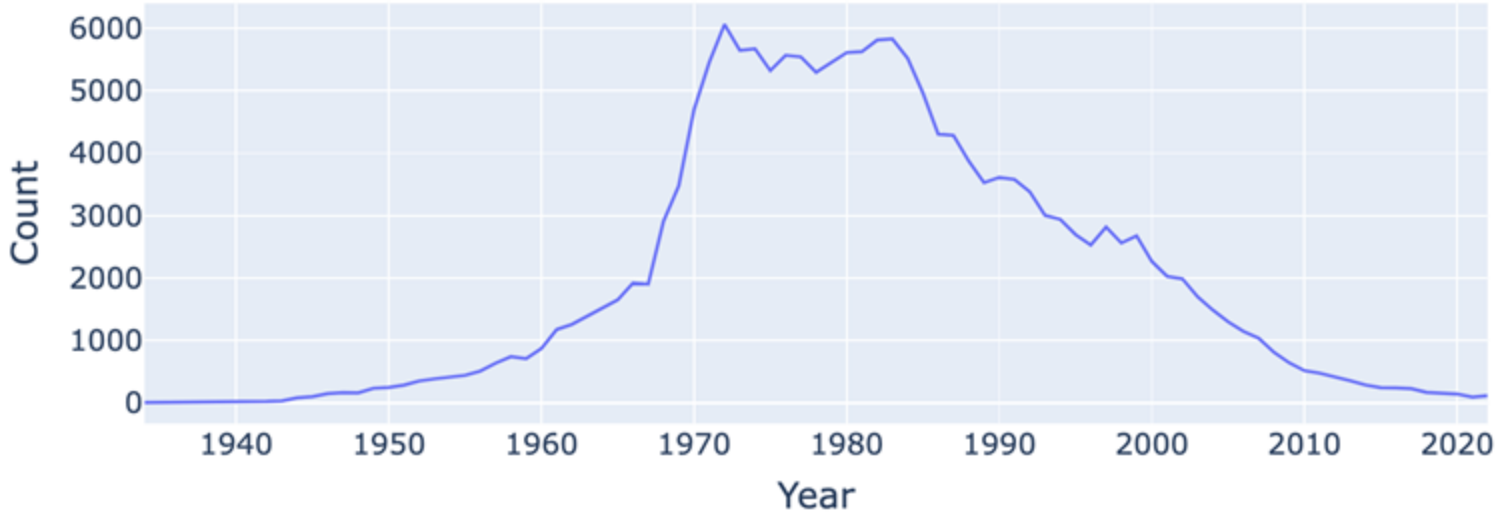
Exemple 2

Objectif: identifier, en Californie, le prénom féminin (sex = "F") dont la popularité a le plus diminué.

```
f_babynames = babynames[babynames["Sex"]=="F"]
```

```
jenn_counts_ser = f_babynames[f_babynames["Name"]=="Jennifer"]["Count"]
```

Nombre de personnes portant le nom `"Jennifer"` nées en Californie par année.



C’est quoi une popularité?

Objectif: Identifier, en Californie, le prénom féminin (sex = "F") dont la popularité a le plus fortement diminué au fil du temps.

Comment mesurer la « baisse de popularité » ?

Nous définissons un indicateur : **Ratio to Peak (RTP)**.

$$RTP = \frac{\textit{Nombre de naissances pour un prénom une année donnée}}{\textit{Nombre maximal de naissances pour ce même prénom (toutes années confondues)}}$$

- **RTP ≈ 1** : popularité proche du pic historique
- **RTP proche de 0** : forte diminution par rapport au pic

Exemple pour Jennifer

- Pic en 1972 : **6 065** naissances
- En 2022 : **114** naissances
- $RTP_{2022} = \frac{114}{6065} \approx 0{,}019$

Calculer le RTP

```
max_jenn = max(jenn_counts_ser)
```

```
6065
```

```
curr_jenn = jenn_counts_ser.iloc[-1]
```

```
114
```



Rappel : **f_babynames** est trié par année dans l'ordre croissant.
.iloc[-1] signifie "récupérer le dernier élément de la `Series`".

```
rtp = curr_jenn / max_jenn
```

```
0.018796372629843364
```

```
def ratio_to_peak(series):
    return series.iloc[-1] / max(series)
```

```
ratio_to_peak(jenn_counts_ser)
```

```
0.018796372629843364
```

Calculer RTP en utilisant `.groupby()`

La méthode `.groupby()` permet de calculer facilement le RTP **pour tous les prénoms en même temps !**

```
rtp_table = f_babynames.groupby("Name")[["Year","Count"]].agg(ratio_to_peak)
```

Name	Year	Count
Aadhini	1.0	1.000000
Aadhira	1.0	0.500000
Aadhya	1.0	0.660000
Aadya	1.0	0.586207
Aahana	1.0	0.269231
...	...	...
Zyanya	1.0	0.466667
Zyla	1.0	1.000000
Zylah	1.0	1.000000
Zyra	1.0	1.000000
Zyrah	1.0	0.833333

13782 rows × 2 columns

```
def ratio_to_peak(series):
    return series.iloc[-1] / max(series)
```

Remarque

L'exécution de l'instruction `.agg()` ci-dessous génère une `TypeError` ou un avertissement, selon la version de `pandas` utilisée. En effet, il est impossible d'effectuer une opération mathématique, comme une division, sur une valeur de type chaîne de caractères, par exemple "CA".

```
f_babynames.groupby("Name").agg(ratio_to_peak)
```

Il est nécessaire de spécifier la colonne numérique à agréger.

```
rtp_table = f_babynames.groupby("Name")[["Count"]].agg(ratio_to_peak)
```

	Count
Name	
Aadhini	1.000000
Aadhira	0.500000
Aadhya	0.660000
Aadya	0.586207
Aahana	0.269231
...	...

Renommer les colonnes après un regroupement

Par defaut, `.groupby()` ne renomme pas les colonnes agrégées. Par exemple, Notre colonne s'appelle toujours "Count", même si elle représente désormais le RTP.

Pour une meilleure lisibilité, nous pouvons renommer "Count" en "Count RTP".

```
rtp_table = f_babynames.groupby("Name")[["Count"]].agg(ratio_to_peak)
rtp_table = rtp_table.rename(columns={"Count":"Count RTP"})
```

Name	Count
Aadhini	1.000000
Aadhira	0.500000
Aadhya	0.660000
Aadya	0.586207
Aahana	0.269231
...	...



Name	Count RTP
Aadhini	1.000000
Aadhira	0.500000
Aadhya	0.660000
Aadya	0.586207
Aahana	0.269231
...	...

Autres résultats

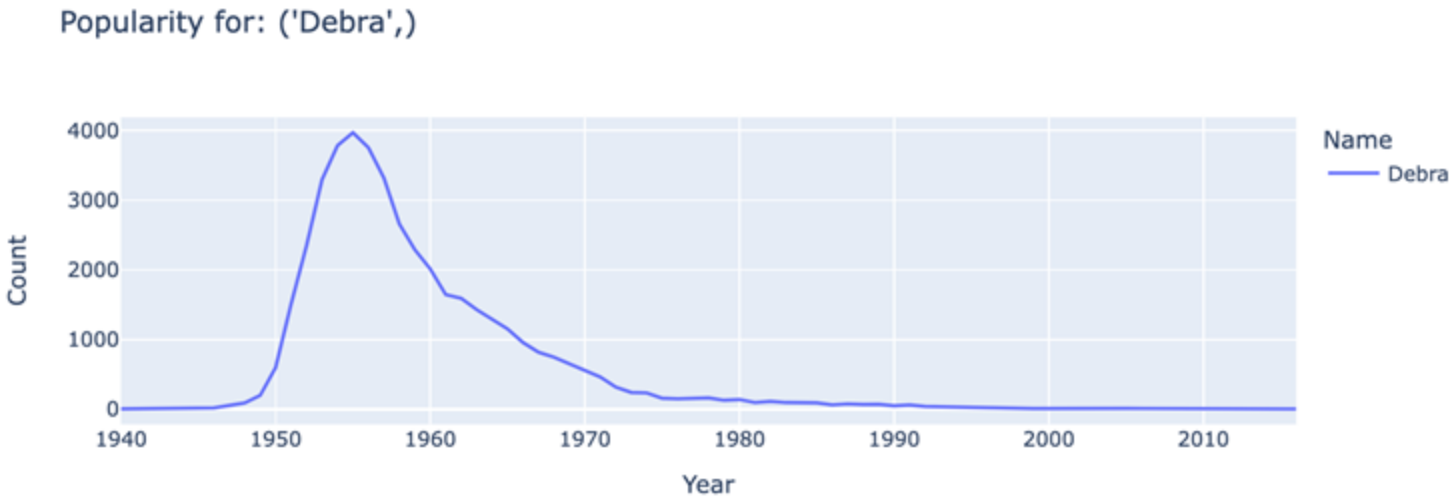
En triant rtp_table, nous pouvons identifier les prénoms dont la popularité a le plus diminué.

```
rtp_table.sort_values("Count RTP")
```

Name	Count RTP
Debra	0.001260
Debbie	0.002815
Carol	0.003180
Tammy	0.003249
Susan	0.003305
...	...
Fidelia	1.000000
Naveyah	1.000000
Finlee	1.000000
Roseline	1.000000
Aadhini	1.000000

13782 rows × 1 columns

```
px.line(f_babynames[f_babynames["Name"]=="Debra"],
                    x="Year", y="Count")
```



Autres résultats

Nous pouvons obtenir la liste des 10 prénoms les moins populaires, puis tracer leur évolution de popularité.

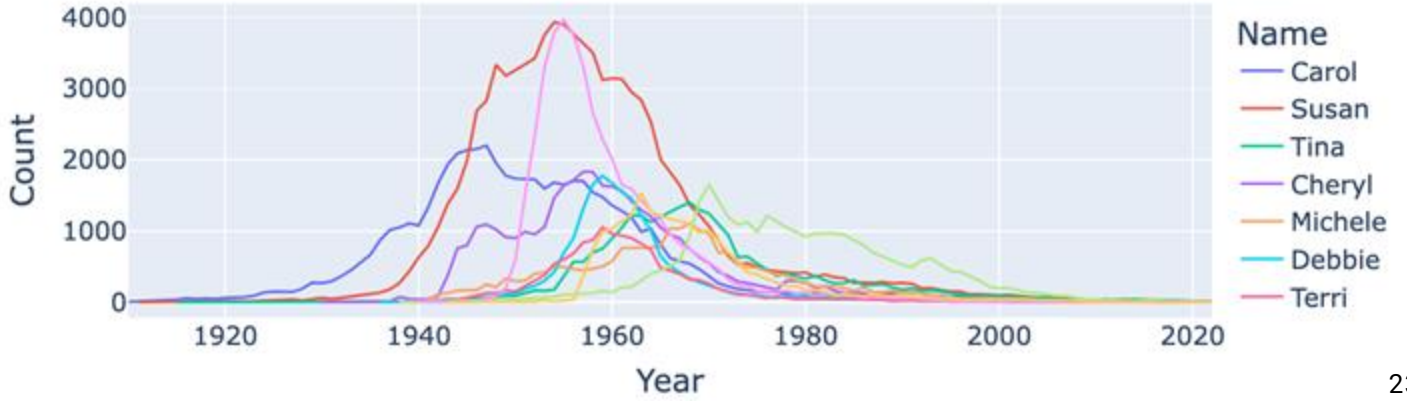
```
top10 = rtp_table.sort_values("Count RTP").head(10).index
```

Name	Count RTP
Debra	0.001260
Debbie	0.002815
Carol	0.003180
Tammy	0.003249
Susan	0.003305
...	...
Fidelia	1.000000
Naveyah	1.000000
Finlee	1.000000
Roseline	1.000000
Aadhini	1.000000

13782 rows × 1 columns

```
Index(['Debra', 'Debbie', 'Carol', 'Tammy', 'Susan', 'Cheryl', 'Shannon',
       'Tina', 'Michele', 'Terri'],
      dtype='object', name='Name')
```

```
px.line(f_babynames[f_babynames["Name"].isin(top10)],
                              x="Year", y="Count", color="Name")
```



`GroupBy` et et méthodes

Le résultat d'une opération `groupby` appliquée à un `DataFrame` est un objet `DataFrameGroupBy` (et non un `DataFrame` !!).

```
grouped_by_year = elections.groupby("Year")
type(grouped_by_year)
```

```
pandas.core.groupby.generic.DataFrameGroupBy
```

À partir d'un objet `DataFrameGroupBy`, on peut utiliser différentes fonctions pour générer des `DataFrames` (ou des `Series`). `agg` n'est qu'une seule option parmi d'autres.

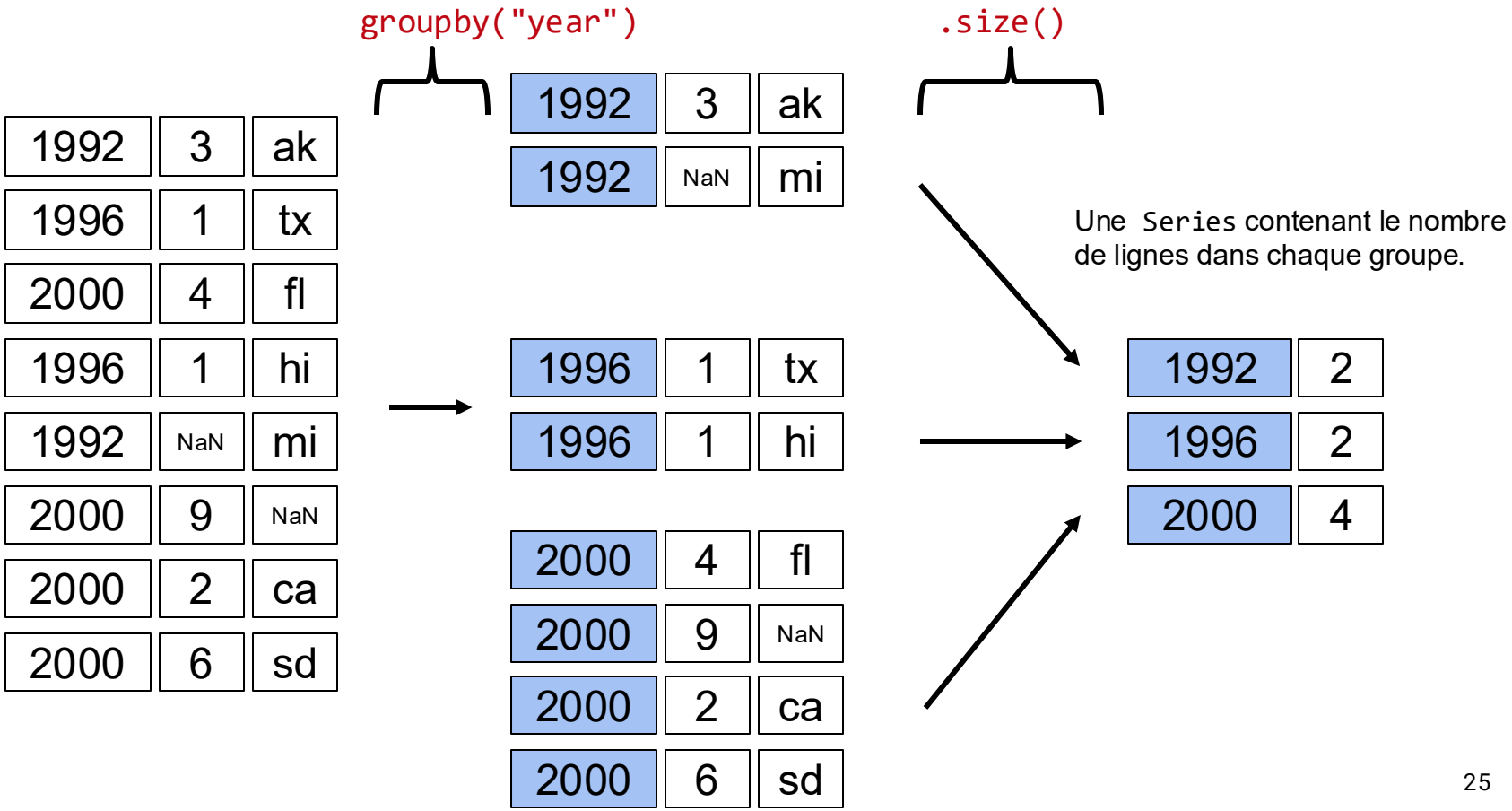
```
df.groupby(col).mean()
df.groupby(col).sum()
df.groupby(col).min()
df.groupby(col).max()
```

```
df.groupby(col).first()
df.groupby(col).last()
df.groupby(col).size()
df.groupby(col).count()
```

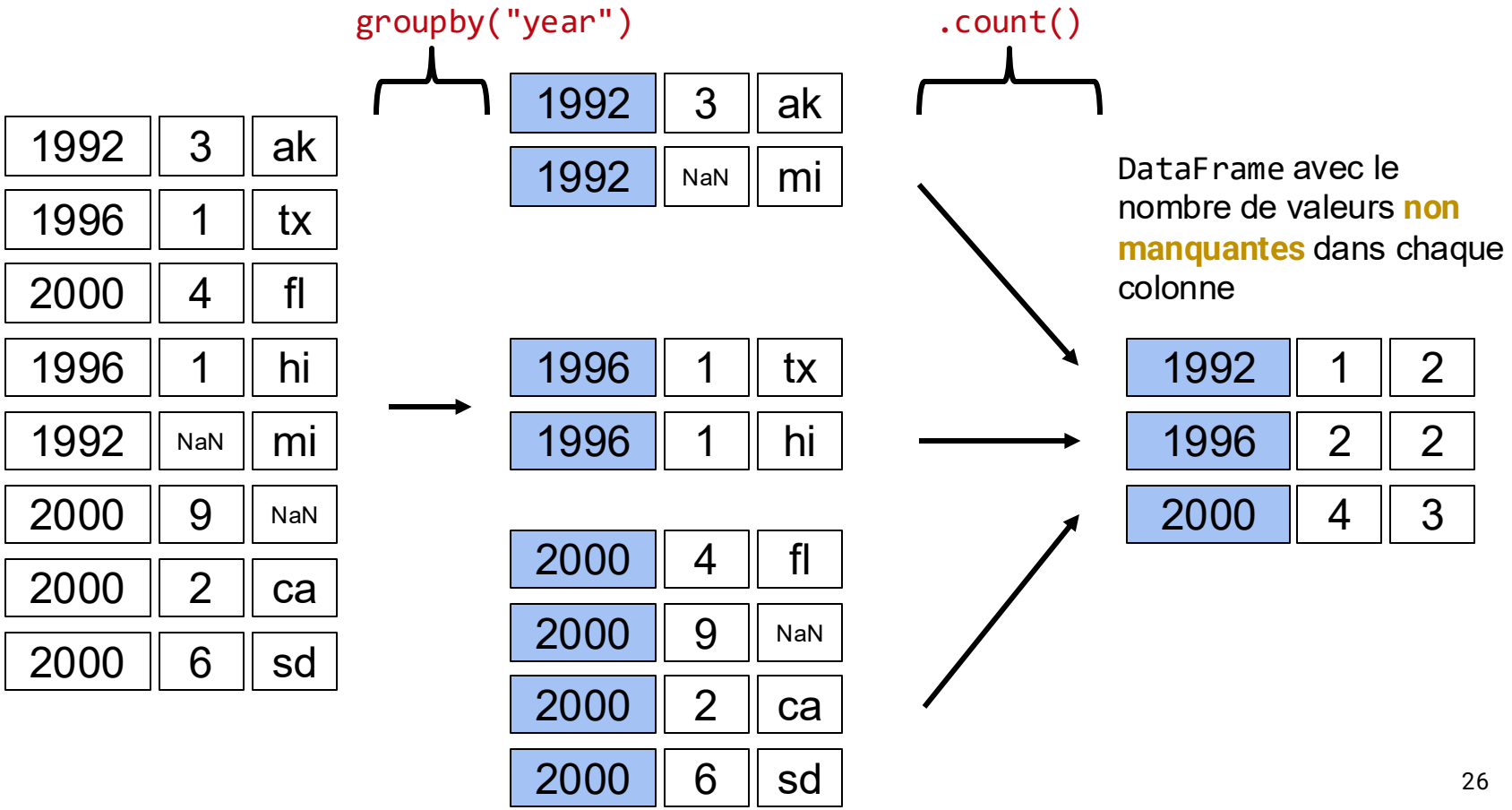
```
df.groupby(col).filter()
```

Voir pandas.pydata.org/docs/reference/groupby.html pour la liste de méthode de `DataFrameGroupBy`.

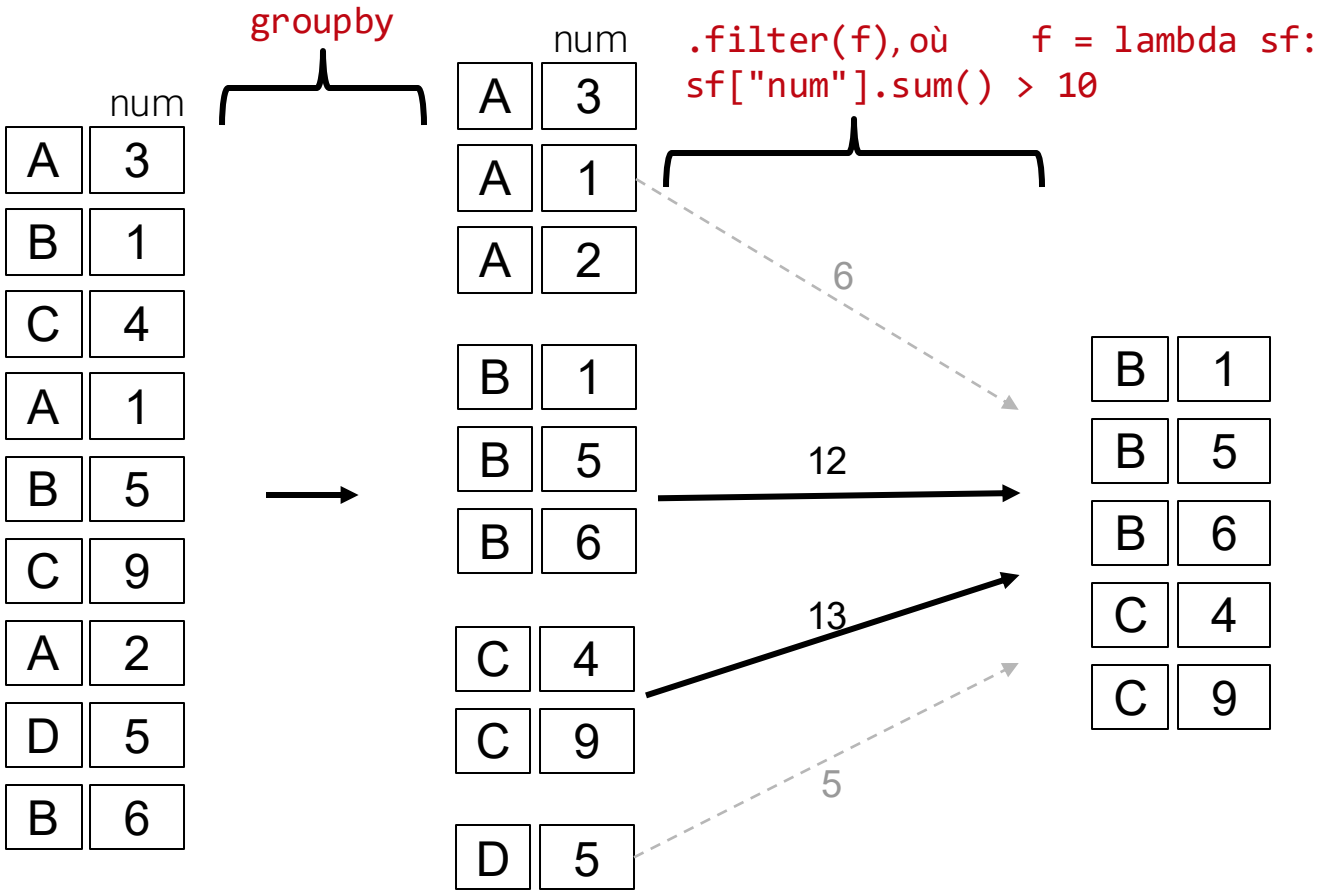
`.size()`


groupby("year")
.size()
1992 3 ak
1996 1 tx
2000 4 fl
1996 1 hi
1992 NaN mi
2000 9 NaN
2000 2 ca
2000 6 sd
1992 3 ak
1992 NaN mi
1996 1 tx
1996 1 hi
2000 4 fl
2000 9 NaN
2000 2 ca
2000 6 sd
Une Series contenant le nombre de lignes dans chaque groupe.
1992 2
1996 2
2000 4

.count()


groupby("year")
.count()
1992 3 ak
1996 1 tx
2000 4 fl
1996 1 hi
1992 NaN mi
2000 9 NaN
2000 2 ca
2000 6 sd
1992 3 ak
1992 NaN mi
1996 1 tx
1996 1 hi
2000 4 fl
2000 9 NaN
2000 2 ca
2000 6 sd
DataFrame avec le nombre de valeurs non manquantes dans chaque colonne
1992 1 2
1996 2 2
2000 4 3

.filter()


groupby
num
A 3
B 1
C 4
A 1
B 5
C 9
A 2
D 5
B 6
num
A 3
A 1
A 2
B 1
B 5
B 6
C 4
C 9
D 5
.filter(f), où f = lambda sf:
sf["num"].sum() > 10
6
12
13
5
B 1
B 5
B 6
C 4
C 9

Filtrage par groupe

Une autre utilisation courante des groupes consiste à filtrer les données.

```
groupby(___).filter(func)
```

- Le filtrage est effectué **par groupe**, et non **par ligne**.
- La méthode `filter` applique la fonction `func` à chaque sous-`DataFrame` correspondant à un groupe (`sf`).
- La fonction `func` doit retourner une seule valeur booléenne `(True ou False)` pour chaque sous-`DataFrame sf.`
    - Si `func` retourne **True** pour un sous-DataFrame sf, **alors toutes les lignes appartenant à ce groupe sont conservées.**
    - Si `func` retourne **False** pour un sous-DataFrame sf, alors **toutes les lignes appartenant à ce groupe sont supprimées.**

Contrairement à .agg(), la colonne utilisée pour le groupement ne devient PAS l'index.

Exemple de Filterage sur la base Elections

Filtrer les années d'élections pour lesquelles le pourcentage maximal de victoire est inférieur à 45 %.

```
elections.groupby("Year").filter(lambda sf: sf["%"].max() < 45)
```

	Year	Candidate	Party	Popular vote	Result	%
23	1860	Abraham Lincoln	Republican	1855993	win	39.699408
24	1860	John Bell	Constitutional Union	590901	loss	12.639283
25	1860	John C. Breckinridge	Southern Democratic	848019	loss	18.138998
26	1860	Stephen A. Douglas	Northern Democratic	1380202	loss	29.522311
66	1912	Eugene V. Debs	Socialist	901551	loss	6.004354
67	1912	Eugene W. Chafin	Prohibition	208156	loss	1.386325
68	1912	Theodore Roosevelt	Progressive	4122721	loss	27.457433
69	1912	William Taft	Republican	3486242	loss	23.218466
70	1912	Woodrow Wilson	Democratic	6296284	win	41.933422
115	1968	George Wallace	American Independent	9901118	loss	13.571218

Question

Nous voulons connaître la meilleure élection de chaque parti.

- Meilleure élection : l'élection au cours de laquelle le pourcentage de votes obtenus est le plus élevé.
- Par exemple, la meilleure élection du Parti démocrate a eu lieu en 1964, avec le candidat Lyndon Johnson remportant 61,3 % des voix.

Party	Year	Candidate	Popular vote	Result	%
American	1856	Millard Fillmore	873053	loss	21.554001
American Independent	1968	George Wallace	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2008	Chuck Baldwin	199750	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	1964	Lyndon Johnson	43127041	win	61.344703

Tentative n°1 : Discussion

Pourquoi l'instruction ci-dessous fait-elle apparaître Woodrow Wilson comme vainqueur de l'élection présidentielle de 2020 ?

```
elections.groupby("Party").max().head(10)
```

Party	Year	Candidate	Popular vote	Result	%
American	1976	Thomas J. Anderson	873053	loss	21.554001
American Independent	1976	Lester Maddox	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2016	Michael Peroutka	203091	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	2020	Woodrow Wilson	81268924	win	61.344703
Democratic-Republican	1824	John Quincy Adams	151271	win	57.210122

Tentative n°1 : Discussion

Pourquoi l'instruction ci-dessous fait-elle apparaître Woodrow Wilson comme vainqueur de l'élection présidentielle de 2020 ?

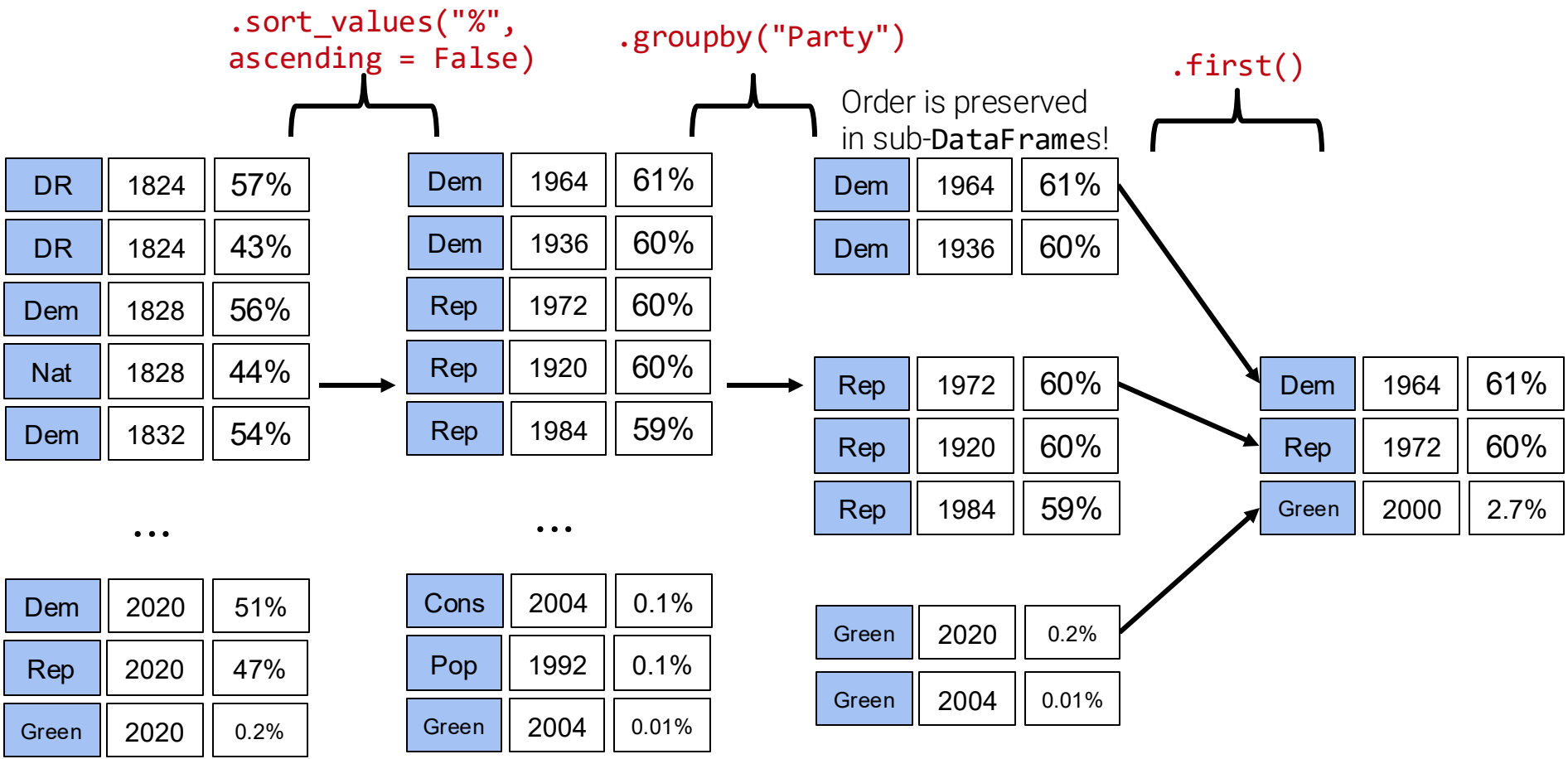
```
elections.groupby("Party").max().head(10)
```

Chaque colonne est traitée séparément.

- Dernière année où les democrates ont participé : 2020
- Nom du candidat le plus "grand" par ordre alphabétique : Woodrow Wilson
- Pourcentage de votes le plus élevé : 61,34 %

Party	Year	Candidate	Popular vote	Result	%
American	1976	Thomas J. Anderson	873053	loss	21.554001
American Independent	1976	Lester Maddox	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2016	Michael Peroutka	203091	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	2020	Woodrow Wilson	81268924	win	61.344703
Democratic-Republican	1824	John Quincy Adams	151271	win	57.210122

Tentative n°2



Tentative n°2

1. Trier le `DataFrame` de manière à ce que les lignes soient classées par ordre décroissant du pourcentage (%).

2. Grouper par Party et prendre le premier élément de chaque sous-`DataFrame`.

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)

elections_sorted_by_percent.groupby("Party").first()
```

	Year	Candidate	Party	Popular vote	Result	%
114	1964	Lyndon Johnson	Democratic	43127041	win	61.344703
91	1936	Franklin Roosevelt	Democratic	27752648	win	60.978107
120	1972	Richard Nixon	Republican	47168710	win	60.907806
79	1920	Warren Harding	Republican	16144093	win	60.574501
133	1984	Ronald Reagan	Republican	54455472	win	59.023326

```
elections_sorted_by_percent
```

Party	Year	Candidate	Popular vote	Result	%
American	1856	Millard Fillmore	873053	loss	21.554001
American Independent	1968	George Wallace	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2008	Chuck Baldwin	199750	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	1964	Lyndon Johnson	43127041	win	61.344703

Autres approches

Using a `lambda` function

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)
elections_sorted_by_percent.groupby("Party").agg(lambda x : x.iloc[0])
```

Using `idxmax` function

```
best_per_party = elections.loc[elections.groupby("Party")["%"].idxmax()]
```

Using `drop_duplicates` function

```
best_per_party2 = elections.sort_values("%").drop_duplicates(["Party"], keep="last")
```

groupby().transform()

1845239

Calculer une statistique par groupe et la reporter sur chaque ligne d'origine

Idée clé : même nombre de lignes

transform() renvoie des valeurs alignées sur les lignes initiales : idéal pour créer une nouvelle colonne.

```
df = pd.DataFrame({
    "departement": ["IT", "IT", "RH", "RH"],
    "salaire": [3000, 5000, 4000, 6000]
})
df["moyenne_dept"] = (
    df.groupby("departement")["salaire"]
      .transform("mean")
)
```

Exemple : chaque employé reçoit la moyenne des salaires de son département.

Exemple de résultat

Département	Salaire	Moy. département
IT	3 000	**4 000**
IT	5 000	**4 000**
RH	4 000	**5 000**
RH	6 000	**5 000**

Résultat : 4 lignes en entrée → 4 valeurs en sortie.

À retenir : transform = résultat aligné sur les observations.

Pandas : groupby().apply()

1845239

Objectif : calculer l'écart entre le salaire maximum et minimum de chaque département.

```
import pandas as pd

df = pd.DataFrame({
    "departement": ["IT", "IT", "RH", "RH"],
    "salaire": [3000, 5000, 4000, 6000]
})

def ecart_salaires(groupe):
    return groupe["salaire"].max() - groupe["salaire"].min()

resultat = df.groupby("departement").apply(ecart_salaires)
print(resultat)
```

Résultat attendu

```
departement
RH    2000
IT    2000
Name: salaire, dtype: int64
```

Chaque groupe produit une seule valeur : max(salaire) – min(salaire).

À retenir : apply() applique une fonction personnalisée à chaque groupe et combine les résultats.

CC BY NC SA

Explorer les objets `DataFrameGroupby`

Voici comment nous pouvons explorer un objet DataFrameGroupBy :

```
grouped_by_party = elections.groupby("Party")
grouped_by_party.groups
```

```
{'American': [22, 126], 'American Independent': [115, 119, 124], 'Anti-Masonic': [6], 'Anti-Monopoly': [38], 'Citize
ns': [127], 'Communist': [89], 'Constitution': [160, 164, 172], 'Constitutional Union': [24], 'Democratic': [2, 4,
8, 10, 13, 14, 17, 20, 28, 29, 34, 37, 39, 45, 47, 52, 55, 57, 64, 70, 74, 77, 81, 83, 86, 91, 94, 97, 100, 105, 10
8, 111, 114, 116, 118, 123, 129, 134, 137, 140, 144, 151, 158, 162, 168, 176, 178, 183], 'Democratic-Republican':
[0, 1], 'Dixiecrat': [103], 'Farmer-Labor': [78], 'Free Soil': [15, 18], 'Green': [149, 155, 156, 165, 170, 177, 18
1, 184], 'Greenback': [35], 'Independent': [121, 130, 143, 161, 167, 174, 185], 'Liberal Republican': [31], 'Liberta
rian': [125, 128, 132, 138, 139, 146, 153, 159, 163, 169, 175, 180], 'Libertarian Party': [186], 'National Democrati
c': [50], 'National Republican': [3, 5], 'National Union': [27], 'Natural Law': [148], 'New Alliance': [136], 'North
ern Democratic': [26], 'Populist': [48, 61, 141], 'Progressive': [68, 82, 101, 107], 'Prohibition': [41, 44, 49, 51,
54, 59, 63, 67, 73, 75, 99], 'Reform': [150, 154], 'Republican': [21, 23, 30, 32, 33, 36, 40, 43, 46, 53, 56, 60, 6
5, 69, 72, 79, 80, 84, 87, 90, 96, 98, 104, 106, 109, 112, 113, 117, 120, 122, 131, 133, 135, 142, 145, 152, 157, 16
6, 171, 173, 179, 182], 'Socialist': [58, 62, 66, 71, 76, 85, 88, 92, 95, 102], 'Southern Democratic': [25], 'State
s' Rights': [110], 'Taxpayers': [147], 'Union': [93], 'Union Labor': [42], 'Whig': [7, 9, 11, 12, 16, 19]}
```

```
grouped_by_party.get_group("Socialist")
```

	Year	Candidate	Party	Popular vote	Result	%
58	1904	Eugene V. Debs	Socialist	402810	loss	2.985897
62	1908	Eugene V. Debs	Socialist	420852	loss	2.850866
66	1912	Eugene V. Debs	Socialist	901551	loss	6.004354
71	1916	Allan L. Benson	Socialist	590524	loss	3.194193

Plan

Pandas, Partie III

- Groupby
- **Tables croisées dynamiques (Pivot tables)**
- Jointure de DataFrame
- Reshaping des données (melt, pivot)

Regrouper les données selon plusieurs colonnes

Nous souhaitons connaître le nombre total de naissances par **sexe** pour chaque **année**. Une approche consiste à regrouper simultanément les deux colonnes concernées :

```
babynames.groupby(["Year", "Sex"])[["Count"]].agg(sum).head(6)
```

		Count
Year	Sex	
1910	F	5950
	M	3213
1911	F	6602
	M	3381
1912	F	9804
	M	8142

Remarque : le **DataFrame** résultant possède un index à plusieurs niveaux, c'est-à-dire que son **index est multidimensionnel**

Pourquoi utiliser [["Count"]] et non ["Count"] ? Les deux fonctionnent ! ["Count"] renvoie une Series.

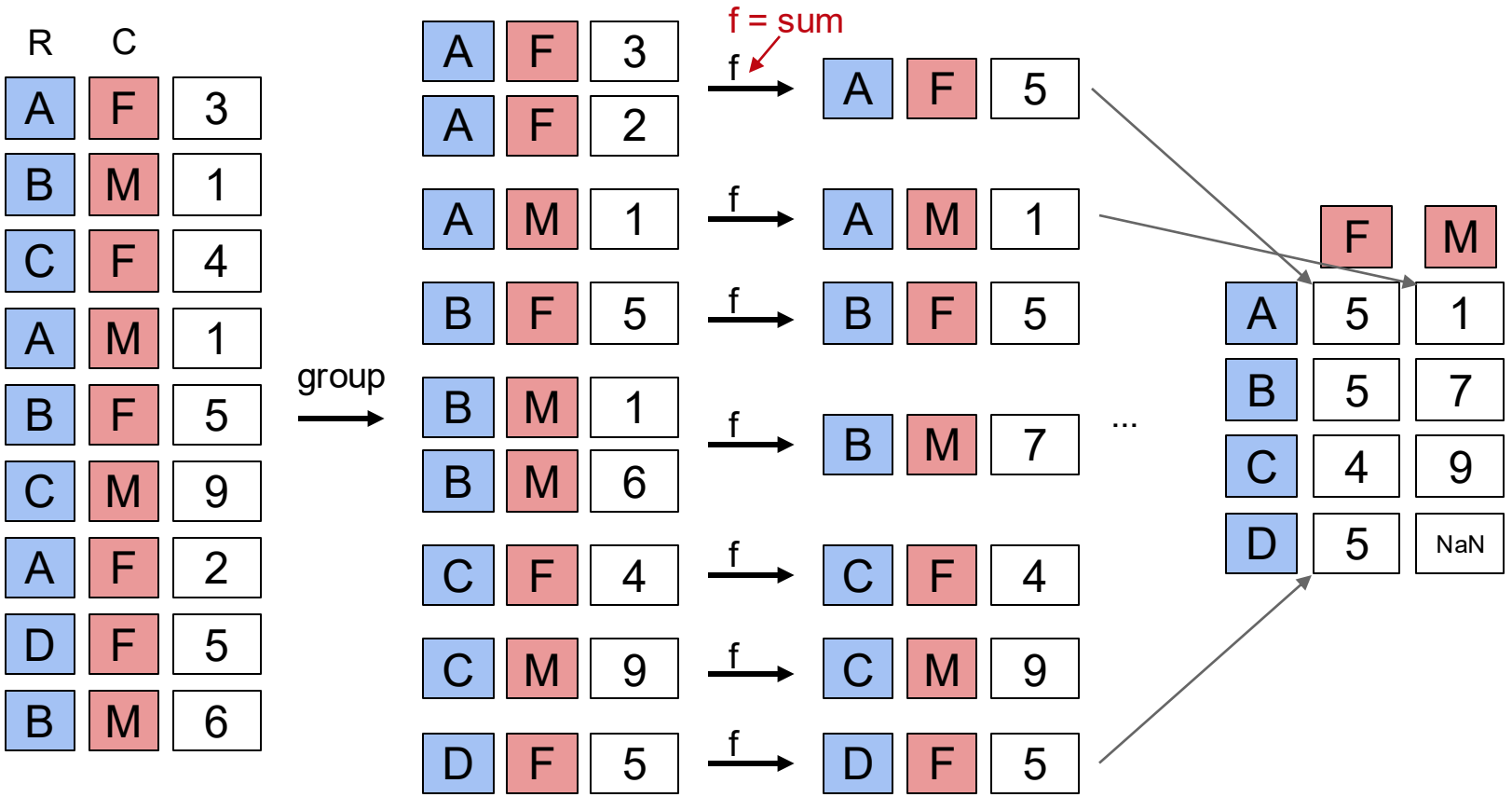
Tables croisées dynamique (Pivot Tables)

Une autre approche consiste à créer un tableau croisé dynamique.

```
babynames_pivot = babynames.pivot_table(
    index = "Year",     # rows (turned into index)
    columns = "Sex",    # column values
    values = ["Count"], # field(s) to process in each group
    aggfunc = np.sum,   # group operation
)
babynames_pivot.head(6)
```

Sex	F	M
Year		
1910	5950	3213
1911	6602	3381
1912	9804	8142
1913	11860	10234
1914	13815	13111
1915	18643	17192

Pivot Table Mechanics



Tableaux croisés dynamiques (Pivot Table) avec plusieurs valeurs

Nous pouvons inclure plusieurs valeurs dans nos tableaux croisés dynamiques.

```
babynames_pivot = babynames.pivot_table(
    index = "Year",     # rows (turned into index)
    columns = "Sex",    # column values
    values = ["Count", "Name"],
    aggfunc = np.max,   # group operation
)
babynames_pivot.head(6)
```

	Count		Name	
Sex	F	M	F	M
Year				
1910	295	237	Yvonne	William
1911	390	214	Zelma	Willis
1912	534	501	Yvonne	Woodrow
1913	584	614	Zelma	Yoshio
1914	773	769	Zelma	Yoshio
1915	998	1033	Zita	Yukio

Autres paramètres utiles:

- margins=True → ajoute les totaux.

Plan

Pandas, Partie III

- Groupby
- Tables croisées dynamiques (Pivot tables)
- **Jointure de DataFrame**
- Reshaping des données (melt, pivot)

Jointure Pandas

On souhaite analyser la popularité des prénoms de candidats présidentiels en 2022.

- **Exemple :** le prénom *Dwight*, porté par Dwight Eisenhower, n'est plus populaire aujourd'hui : en 2022, seuls 5 nouveau-nés en Californie ont reçu ce prénom.

Pour commencer à traiter ce problème, il est nécessaire d'effectuer une **jointure de jeux de données.**

Création de la Table 1 : Prénoms de bébés en 2022

On commence par appliquer un filtrage sur le jeu de données (de la base `babynames`) afin de conserver uniquement les lignes correspondant à l'année 2022.

```
babynames_2022 = babynames[babynames["Year"] == 2022]
babynames_2022
```

	State	Sex	Year	Name	Count
235835	CA	F	2022	Olivia	2178
235836	CA	F	2022	Emma	2080
235837	CA	F	2022	Camila	2046
235838	CA	F	2022	Mia	1882
235839	CA	F	2022	Sophia	1762
235840	CA	F	2022	Isabella	1733
235841	CA	F	2022	Luna	1516
235842	CA	F	2022	Sofia	1307
235843	CA	F	2022	Amelia	1289
235844	CA	F	2022	Gianna	1107

Création de la Table 2 : Présidents avec leurs prénoms

On isole également les prénoms des candidats afin de disposer d'une clé commune entre les deux tables

```
elections["First Name"] = elections["Candidate"].str.split().str[0]
```

	Year	Candidate	Party	Popular vote	Result	%	First Name
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	Andrew
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	John
2	1828	Andrew Jackson	Democratic	642806	win	56.203927	Andrew
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073	John
4	1832	Andrew Jackson	Democratic	702735	win	54.574789	Andrew
...	...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629	Donald
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772	Kamala
184	2024	Jill Stein	Green	861155	loss	0.554864	Jill
185	2024	Robert Kennedy	Independent	756383	loss	0.487357	Robert
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895	Chase

187 rows × 7 columns

Jointure des deux Tables: Deux Options

```
merged = pd.merge(left = elections, right = babynames_2022,
                  left_on = "First Name", right_on = "Name")

merged = elections.merge(right = babynames_2022,
                  left_on = "First Name", right_on = "Name")
```

	Year_x	Candidate	Party	Popular vote	Result	%	First Name	State	Sex	Year_y	Name	Count
75	1892	Benjamin Harrison	Republican	5176108	loss	42.984101	Benjamin	CA	M	2022	Benjamin	1524
73	1884	Benjamin Butler	Anti-Monopoly	134294	loss	1.335838	Benjamin	CA	M	2022	Benjamin	1524
74	1888	Benjamin Harrison	Republican	5443633	win	47.858041	Benjamin	CA	M	2022	Benjamin	1524
45	1880	James Garfield	Republican	4453337	win	48.369234	James	CA	M	2022	James	1086
43	1880	James B. Weaver	Greenback	308649	loss	3.352344	James	CA	M	2022	James	1086
...	...	...	...	...	...	...	...	...	...	...	...	...
115	1964	Lyndon Johnson	Democratic	43127041	win	61.344703	Lyndon	CA	M	2022	Lyndon	6
92	1912	Woodrow Wilson	Democratic	6296284	win	41.933422	Woodrow	CA	M	2022	Woodrow	6
93	1916	Woodrow Wilson	Democratic	9126868	win	49.367987	Woodrow	CA	M	2022	Woodrow	6
76	1888	Clinton B. Fisk	Prohibition	249819	loss	2.196299	Clinton	CA	M	2022	Clinton	6
145	2016	Darrell Castle	Constitution	203091	loss	0.149640	Darrell	CA	M	2022	Darrell	5

152 rows × 12 columns

From here, use your new `.groupby()` tools!

Plan

Pandas, Partie III

- Groupby
- Tables croisées dynamiques (Pivot tables)
- Jointure de DataFrame
- **Reshaping des données (melt, pivot)**

Reshaping des données avec melt()

Parfois, il faut passer d'un tableau large à un tableau long. C'est ce que fait pd.melt()

```
df = pd.DataFrame({
    'Produit': ['A', 'B', 'C'],
    '2022': [100, 150, 200],
    '2023': [120, 170, 210]
})

df_long = df.melt(
    id_vars='Produit',
    var_name='Année',
    value_name='Ventes'
)
```

	Produit	2022	2023
0	A	100	120
1	B	150	170
2	C	200	210

	Produit	Année	Ventes
0	A	2022	100
1	B	2022	150
2	C	2022	200
3	A	2023	120
4	B	2023	170
5	C	2023	210

Résultat :

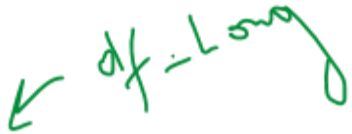
- Colonnes → Produit | Année | Ventes
- Chaque année devient une ligne.

Reshaping des données avec pivot()

Objectif : Repasser du format long au format large.

```
df_wide = df_long.pivot(
    index='Produit',
    columns='Année',
    values='Ventes'
)
```

	Produit	Année	Ventes
0	A	2022	100
1	B	2022	150
2	C	2022	200
3	A	2023	120
4	B	2023	170
5	C	2023	210



Résultat :

- Colonnes → 2022 | 2023
- Chaque année redevient une colonne.

Produit \ Année	2022	2023
A	100	120
B	150	170
C	200	210

Remarque :

- pivot() nécessite des combinaisons uniques.
- Sinon utiliser pivot_table() avec agrégation.

Pivot_table() vs pivot()

```
df_long_extra = pd.concat([
    df_long,
    pd.DataFrame({"Produit":["A"]
, "Année":["2022"],
"Ventes":[130]})
])
```

	Produit	Année	Ventes
0	A	2022	100
1	B	2022	150
2	C	2022	200
3	A	2023	120
4	B	2023	170
5	C	2023	210
0	A	2022	130

```
df_long_extra.pivot(
    index="Produit",
    columns="Année",
    values="Ventes"
)
```

```
ValueError: Index contains duplicate entries, cannot reshape
```

```
df_long_extra.pivot_table(
    index="Produit",
    columns="Année",
    values="Ventes",
    aggfunc="sum"
)
```

Année	2022	2023
Produit		
A	230	120
B	150	170
C	200	210